

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No.: S040	Name: Omkar Prakash Sawant
Class: TY.IT	Batch: 1
Date of Assignment: 14/07/2026	Date/Time of Submission: 14/07/2026

- a) Create methods add(), multiply(), subtract(), divide() with suitable parameters and call these methods using concept of C# delegate.**

Algorithm:

Step 1 - Start.

Step 2 - Declare a delegate with two integer parameters.

Step 3 - Create methods:

Add(int a, int b)

Subtract(int a, int b)

Multiply(int a, int b)

Divide(int a, int b)

Step 4- In the Main() method:

Create a delegate object.

Assign the Add() method to the delegate and call it.

Assign the Subtract() method to the delegate and call it.

Assign the Multiply() method to the delegate and call it.

Assign the Divide() method to the delegate and call it.

Step 5 - Display the results.

Step 6 - Stop

Code:

```
using System;
delegate void operation(inta,intb);
class calculator
{
    public void Add(inta,intb)
    {
        console.writeline("Additon="+ (a+b));
    }
    public void multiply(inta,intb)
    {
        console.writeline("multiply="* (a*b));
    }
    public void subtraction(inta,intb)
    {

```

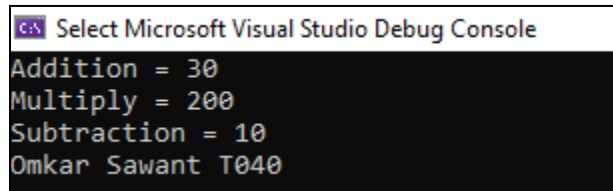
```
        console.WriteLine("subtraction=" + (a-b));
    }
}
public class helloworld
{
    static void main(string[] args)
    {
        calculator c = new calculator();
        operation obj;

        obj=c.Add;
        obj(20,10);

        obj=c.multiply;
        obj(20,10);

        obj=c.subtraction;
        obj(20,10);
        Console.WriteLine("Omkar Sawant T040");
    }
}
```

Output:



The screenshot shows the Microsoft Visual Studio Debug Console with the following output:

```
Addition = 30
Multiply = 200
Subtraction = 10
Omkar Sawant T040
```

b) Write a program using multicast delegate.

Algorithm:

Step 1 - Start.

Step 2- Declare a delegate with no parameters.

Step 3 - Create three methods (e.g., Welcome (), Learn (), ThankYou ()).

Step 4 - In the Main () method:

 Create a delegate object.

 Assign the first method to the delegate.

 Add the remaining methods using the += operator.

 Invoke the delegate object.

Step 5 - All methods execute one after another.

Step 6 - Stop.

Code:

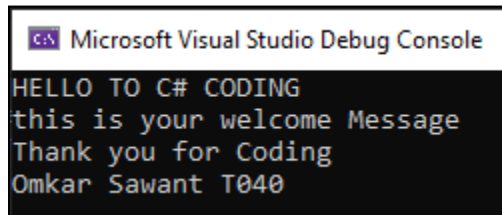
```
using System;
delegate void Message();
class Msg_Print
{

```

Omkar Prakash Sawant T040

```
public void Hello()
{
    Console.WriteLine("HELLO TO C# CODING");
}
public void Msg()
{
    Console.WriteLine("this is your welcome Message");
}
public void Thank()
{
    Console.WriteLine("Thank you for Coding");
}
}
public class Program
{
    static void Main(string[] args)
    {
        Msg_Print mp = new Msg_Print();
        Message msg = mp.Hello;
        msg += mp.Msg;
        msg += mp.Thank;
        msg();
        Console.WriteLine("Omkar Sawant T040");
    }
}
```

Output:



The screenshot shows the Microsoft Visual Studio Debug Console with the following output:

```
HELLO TO C# CODING
this is your welcome Message
Thank you for Coding
Omkar Sawant T040
```

c) Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Algorithm:

Step 1 - Start.

Step 2 - Create a class BankAccount.

Step 3 - Declare private data members:

AccountNumber

Balance

Step 4 - Create a property for Balance.

Step 5 - In the set accessor:

Check whether the assigned value is greater than or equal to 0.

If yes, store the value in Balance.

Otherwise, display an error message: "Balance cannot be negative."

Step 6 - Create an object of the BankAccount class in the Main () method.

Step 7 - Assign the account number.

Step 8 - Assign a valid balance and display the account details.

Step 9 - If a negative balance is entered, display the validation message.

Step 10 - Stop.

Code:

```
using System;
```

```
class Bank
```

```
{
```

```
    public int Acc_No { get; set; }
```

```
    private double bal;
```

```
    public double Bal
```

```
    {
```

```
        get
```

```
        {
```

```
            return bal;
```

```
        }
```

```
        set
```

```
        {
```

```
            if (value >= 0)
```

```
            {
```

```
                bal = value;
```

```
            }
```

```
            else
```

```
            {
```

```
                Console.WriteLine("Balance cannot be Negative");
```

```
            }
```

```
        }
```

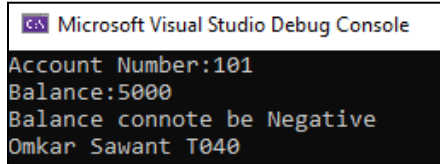
```
    }
```

```
}
```

Omkar Prakash Sawant T040

```
class Program
{
    static void Main(string[] args)
    {
        Bank b=new Bank();
        b.Acc_No = 101;
        b.Bal = 5000;
        Console.WriteLine("Account Number:" + b.Acc_No);
        Console.WriteLine("Balance:"+b.Bal);
        b.Bal = -1000;
        Console.WriteLine("Omkar Sawant T040");
    }
}
```

Output:



Microsoft Visual Studio Debug Console

```
Account Number:101
Balance:5000
Balance connote be Negative
Omkar Sawant T040
```